

Aegis OS Security Threat Model Analysis

Omar Dorgham & Mariam El-Kholey
E-JUST

May 18, 2026

1 Executive Summary

Aegis OS is a hybrid security and insurance platform that combines off-chain AI analysis, a notarization pipeline, and on-chain smart contracts to support cyber incident reporting, policy management, claim adjudication, and immutable audit logging. In operational terms, the system ingests alerts and claim data, processes them through backend agents, anchors the resulting evidence into Merkle roots, and writes the corresponding commitments to blockchain state for later verification.

This analysis uses STRIDE as the primary threat enumeration framework and a likelihood-impact risk assessment to classify each threat. The current security posture is best described as *moderate with meaningful compensating controls*: the design includes ReentrancyGuard, checks-effects-interactions discipline, domain-separated Merkle hashing, atomic file writes, and a single-worker constraint for sequential IDs. At the same time, several risks remain design-dependent rather than fully eliminated, including governance without a timelock, backend trust assumptions, sequence integrity reliance on deployment discipline, and timestamp sensitivity on chain.

Overall, the residual risk is acceptable for a single-machine, localhost-only academic deployment, provided the documented constraints are preserved and the remaining medium-risk items are accepted as intentional trade-offs for development velocity. This acceptance is conditional, not permanent: if the system is exposed beyond the local lab environment, the governance and operational controls should be upgraded before production use.

2 Scope and Assumptions

2.1 In Scope

- All seven smart contracts: `AuditRegistry`, `ClaimsProcessor`, `CyberToken`, `DailyReportRegistry`, `Governance`, `PolicyEngine`, and `PostureRegistry`.
- The backend notarizer and AI agents responsible for alert analysis, anomaly detection, and fraud analysis.
- The frontend MetaMask integration used for wallet connection and transaction signing.
- The SQLite database that stores AI decisions, alerts, claims, summaries, and system actions.

- The Merkle tree implementation used to batch and notarize event evidence.

2.2 Out of Scope

- Hardhat node configuration, except where it affects the threat model of the local deployment.
- LLM provider security, including Google Gemini service-side controls.
- Operating system hardening, host compromise prevention, and local privilege escalation resistance.
- Network infrastructure security, including routers, switches, VPNs, and public Internet exposure.

2.3 Assumptions

- The system is deployed on a single machine for development or demonstration purposes.
- The deployer wallet and local operator are trusted to initialize the stack correctly.
- The application is not publicly exposed on the Internet.
- The Hardhat node is available only on localhost and is not intended to serve untrusted remote clients.

3 Attack Surface Analysis

3.1 Smart Contracts

Solidity automatically exposes public state variables as getter functions, so the attack surface includes both explicit `external` functions and compiler-generated public accessors.

3.1.1 AuditRegistry

3.1.2 ClaimsProcessor

3.1.3 CyberToken

3.1.4 DailyReportRegistry

3.1.5 Governance

3.1.6 PolicyEngine

3.1.7 PostureRegistry

3.2 Backend API

The backend currently exposes no application-level authentication middleware; security depends primarily on local deployment boundaries, trusted clients, and validation performed by the request models and downstream agents.

Callable surface	Who can call	State impact	Value transfer
<code>logAction(bytes32, bytes32, bytes32, RiskLevel)</code>	Any external caller	Appends a new audit entry and emits <code>ActionLogged</code>	None
<code>getActionCountByTimeRange(uint256, uint256)</code>	Any external caller	Read-only query	None
<code>getEntryCount()</code>	Any external caller	Read-only query	None
<code>getEntry(uint256)</code>	Any external caller	Read-only query	None
<code>entries()</code> getter	Any external caller	Read-only array access	None
<code>owner()</code> getter	Any external caller	Read-only owner lookup	None

Callable surface	Who can call	State impact	Value transfer
<code>fileClaim(uint256, string, uint256)</code>	Any policy holder with an active policy	Creates a new claim and increments <code>claimCount</code>	None
<code>recordFraudScore(uint256, uint8, bytes32)</code>	Backend system only	Updates fraud score and report hash for a pending claim	None
<code>processClaim(uint256)</code>	Backend system only	Sets verdict, payout percentage, and processed timestamp	Transfers CIT to claimant if approved or partially approved
<code>getClaim(uint256)</code>	Any external caller	Read-only query	None
<code>postureRegistry()</code> getter	Any external caller	Read-only dependency lookup	None
<code>policyEngine()</code> getter	Any external caller	Read-only dependency lookup	None
<code>auditRegistry()</code> getter	Any external caller	Read-only dependency lookup	None
<code>cyberToken()</code> getter	Any external caller	Read-only dependency lookup	None
<code>claimCount()</code> getter	Any external caller	Read-only counter access	None
<code>backendSystem()</code> getter	Any external caller	Read-only backend address access	None

3.3 Frontend

- MetaMask wallet connection is the only wallet onboarding path. The frontend requests accounts through `eth_requestAccounts` and reloads on account or chain changes.
- Transaction signing occurs through the connected provider and signer; the application does not store private keys locally.

Callable surface	Who can call	State impact	Value transfer
<code>mint(address,uint256)</code>	Owner only	Mints additional CIT supply	Increases token supply to recipient
<code>transfer(address,uint256)</code>	Any token holder	Updates balances	Moves CIT between accounts
<code>transferFrom(address, address, uint256)</code>	Approved spender	Updates balances and allowance	Moves CIT between accounts
<code>approve(address,uint256)</code>	Any token holder	Sets allowance	No direct transfer
<code>allowance(address, address)</code>	Any external caller	Read-only query	None
<code>balanceOf(address)</code>	Any external caller	Read-only query	None
<code>totalSupply()</code>	Any external caller	Read-only query	None
<code>name(), symbol(), decimals()</code>	Any external caller	Read-only query	None
<code>owner(), transferOwnership(address), renounceOwnership()</code>	Owner-related callers	Changes ownership metadata	None

Callable surface	Who can call	State impact	Value transfer
<code>submitDailyReport(bytes32,uint256, uint8)</code>	Backend system only	Records the report for the current day and increments <code>currentDay</code>	None
<code>getReport(uint256)</code>	Any external caller	Read-only query	None
<code>getCurrentDay()</code>	Any external caller	Read-only query	None
<code>dailyReports(uint256)</code>	Any external caller	Read-only mapping access	None
<code>currentDay()</code> getter	Any external caller	Read-only counter access	None
<code>backendSystem()</code> getter	Any external caller	Read-only backend address access	None

- Private key handling is delegated to MetaMask. The web application only sees the selected address and signed transaction results.
- User input validation is partial: some fields are constrained by form controls, but most semantic validation is deferred to the smart contracts and backend services.

3.4 Data Flows

- Alert → Queue → Merkle tree → Blockchain: security events are queued, batched, hashed, and anchored on chain.
- Claim filing → Smart contract → Token transfer: the user submits a claim, backend analysis runs, and approved payouts are executed in **CyberToken**.
- Off-chain reasoning → On-chain hash anchor: AI reasoning artifacts are persisted in SQLite and then committed as hashes to support later verification.

Callable surface	Who can call	State impact	Value transfer
<code>addSigner(address)</code>	Owner only	Adds an authorized signer	None
<code>removeSigner(address)</code>	Owner only	Removes an authorized signer	None
<code>requestApproval(uint256)</code>	Any external caller	Marks an audit entry as pending approval	None
<code>approve(uint256)</code>	Authorized signer only	Moves the entry from pending to approved	None
<code>reject(uint256)</code>	Authorized signer only	Clears the pending flag	None
<code>isApproved(uint256)</code>	Any external caller	Read-only query	None
<code>isPending(uint256)</code>	Any external caller	Read-only query	None
<code>owner()</code> , <code>auditRegistry()</code>	Any external caller	Read-only accessors	None

Callable surface	Who can call	State impact	Value transfer
<code>setPolicy(address, string[], bool[], uint256[], uint8[])</code>	Insurer only	Replaces a company policy and activates it	None
<code>getPolicy(address)</code>	Any external caller	Read-only query	None
<code>isPolicyActive(address)</code>	Any external caller	Read-only query	None
<code>insurer() getter</code>	Any external caller	Read-only dependency lookup	None

Callable surface	Who can call	State impact	Value transfer
<code>recordSnapshot(bytes32, uint8)</code>	Any external caller	Appends a snapshot for <code>msg.sender</code>	None
<code>getLastSnapshot(address)</code>	Any external caller	Read-only query	None
<code>getSnapshotCount(address)</code>	Any external caller	Read-only query	None
<code>getSnapshotByIndex(address, uint256)</code>	Any external caller	Read-only query	None
<code>getSecurityHygieneScore(address)</code>	Any external caller	Read-only query	None

4 Threat Identification and Analysis

4.1 T1: Reentrancy Attack on `ClaimsProcessor.processClaim()`

4.1.1 Threat Description

A malicious contract can attempt to reenter `processClaim()` during the `CyberToken.transfer()` call, before the claim status has been finalized. If the state were updated after the token transfer, the same claim could be paid more than once.

Endpoint	Auth	Validation	Malicious input handling
POST /api/analyze-alert	None	Pydantic shape validation	Untrusted text is forwarded to the security agent; prompt injection or oversized payloads may distort reasoning or increase load
POST /api/ analyze-daily-logs	None	Pydantic validation for daily_logs	Malicious log content may poison analysis, but the payload remains isolated to backend processing and logging
POST /api/analyze-claim	None	Pydantic validation for claim fields	Crafted claim text may bias fraud analysis or create noisy audit output
GET /api/sequence-status	None	No input body	Safe read-only endpoint; abuse is limited to traffic volume
GET /api/verify-event/ {event_hash}	None	Path parameter only	Invalid or nonexistent hashes return 404; malformed hashes are treated as lookup failures
GET /api/db-dump	None	No input body	Returns hashed AI decision material; exposure is sensitive but read-only
GET /	None	No input body	Health check only; little direct attack value
GET /api/activity-feed	None	Optional limit parameter	Large limits increase database work and response size
GET /api/ai-decisions	None	Optional limit parameter	Large limits increase database work and response size
GET /api/claim-analysis/ {claim_id}	None	Integer path parameter	Invalid IDs return 404, but enumeration may reveal claim existence patterns
GET /api/alerts	None	Optional limit parameter	Large limits increase database work and response size
GET /api/system-stats	None	No input body	Low-risk read-only aggregation endpoint

4.1.2 Attack Vector

1. The attacker deploys `MaliciousAttacker.sol`.
2. The attacker files or targets a claim that remains in `PENDING` state.
3. The attacker triggers `processClaim(claimId)`.
4. During `CyberToken.transfer()`, the malicious contract receives control through an external call path.
5. The fallback path attempts to reenter `processClaim(claimId)`.

6. The repeated call would succeed only if the claim had not yet been locked by state changes.
7. Without defenses, the payout could execute more than once.

4.1.3 Likelihood

LOW

- The attacker must deploy a contract on the same chain.
- The attacker must possess a valid claim path and reach the backend-controlled processing branch.
- The claim processor already restricts execution to the backend and applies `nonReentrant`.

4.1.4 Impact

CRITICAL

- Repeated claim payouts could drain contract liquidity.
- Claim settlement integrity would be violated.
- The chain of trust between fraud scoring and payout execution would be corrupted.

4.1.5 Mitigation: OpenZeppelin ReentrancyGuard

```
function processClaim(uint256 claimId)
    external
    nonReentrant
    returns (ClaimDecision memory)
{
    // function body
}
```

The modifier establishes a lock before execution begins. If the call stack attempts to reenter the function, the guard detects the active lock and reverts the transaction.

4.1.6 Mitigation: Checks-Effects-Interactions Pattern

Within `processClaim()` the code follows a disciplined order:

1. **Checks:** validate the claim exists, is pending, and has a recorded fraud score.
2. **Effects:** write the verdict, payout percentage, and processed timestamp before any external transfer.
3. **Interactions:** call `cyberToken.transfer()` only after the state is already locked.

4.1.7 Mitigation Validation

The included attacker contract attempts recursive reentry during the external call path. The expected result is a revert with the OpenZeppelin reentrancy error. In a regression test, this should be treated as a pass condition when the transaction reverts for the documented reason.

4.1.8 Residual Risk

LOW

- The implementation combines a standard audited guard with CEI ordering.
- No known bypass exists for the documented control combination in this code path.
- Residual risk remains only if downstream dependencies are replaced with nonstandard token logic.

4.2 T2: Flash Loan Governance Attack

4.2.1 Threat Description

An attacker borrows a large quantity of voting power through a flash loan, uses the temporary balance to influence governance, executes an approval or rejection action, and repays the loan within the same transaction window.

4.2.2 Attack Vector

1. The attacker acquires a large borrowed balance.
2. A governance function checks token balance or balance-derived authority.
3. The attacker uses the temporary balance to satisfy the voting threshold.
4. A malicious action is approved or executed.
5. The borrowed tokens are repaid in the same block.

4.2.3 Likelihood

MEDIUM

- The attack requires an external flash loan source that supports the relevant token or voting surface.
- The current architecture does not include a timelock or snapshot-based voting model.
- If deployed beyond the local lab environment, the attack becomes materially easier to stage.

4.2.4 Impact

HIGH

- Unauthorized governance actions may be approved.
- Fraudulent claims or policy changes could be executed.
- The governance layer would no longer represent durable stakeholder intent.

4.2.5 Current Mitigation

NOT IMPLEMENTED

- `CyberToken` does not embed flash-loan logic.
- `Governance.sol` does not enforce a timelock.
- Voting is not snapshot-based in the current design.

4.2.6 Future Mitigation: Time-Locked Governance

```
mapping(bytes32 proposalId => uint256 executeAt) public executions;

function requestApproval(uint256 actionId) external returns (bytes32 proposalId) {
    proposalId = keccak256(abi.encode(actionId, block.timestamp));
    executions[proposalId] = block.timestamp + 7 days;
}

function executeApproval(bytes32 proposalId) external {
    require(block.timestamp >= executions[proposalId], "Timelock active")
}
```

A seven-day delay prevents a flash loan from surviving long enough to manipulate governance outcomes.

4.2.7 Residual Risk

MEDIUM (implementation-dependent)

- The present design intentionally prioritizes fast iteration over hardened governance.
- Snapshot voting or a timelock would reduce the risk significantly.
- Until then, the governance surface should be treated as unsuitable for adversarial public deployment.

4.3 T3: Merkle Leaf Second-Preimage Attack

4.3.1 Threat Description

An attacker may try to present an internal Merkle node hash as if it were a real leaf. Without domain separation, the verifier might accept a fabricated leaf proof that reconstructs the same root.

4.3.2 Attack Vector

1. Legitimate events are hashed into leaves.
2. Internal nodes are computed from child hashes.
3. The attacker reuses an internal node hash as a purported event hash.
4. The attacker submits a proof path to the verifier.
5. If leaves and internal nodes are not domain separated, the proof may verify.

4.3.3 Likelihood

LOW

- The implementation uses domain-separated hashing.
- Exploitation would require either a defective implementation or a later regression.

4.3.4 Impact

CRITICAL

- Fabricated evidence could be treated as genuine.
- Audit trails and compliance records would lose integrity.
- Downstream policy or payout logic based on event counts could be manipulated.

4.3.5 Mitigation: Domain Separation

```
# Leaf hashing
leaf_hash = hashlib.sha256(b"\x00" + event_data.encode()).digest()

# Node hashing
node_hash = hashlib.sha256(b"\x01" + left_child + right_child).digest()
```

The distinct prefixes ensure that a leaf hash and an internal node hash live in separate cryptographic domains.

4.3.6 Verification

The test harness should construct a tree, then attempt to verify a fabricated event using an internal node as if it were a leaf. The expected outcome is failure, which indicates that the domain separation prevents the second-preimage substitution.

4.3.7 Residual Risk

LOW

- The cryptographic construction is mathematically sound.
- The implementation matches the documented prefix scheme.
- Residual risk is limited to future code changes that remove or weaken the domain separator.

4.4 T4: Event Dropping During Notarization Window

4.4.1 Threat Description

During the interval between event creation and Merkle batch commitment, an internal operator could delete queued events before notarization. Those events would never appear on chain, even though they were originally generated.

4.4.2 Attack Vector

1. A security event is produced and pushed into the queue.
2. The attacker gains backend-level access to the queue or its runtime.
3. The attacker removes the event before batch processing.
4. The notarizer commits a batch that omits the deleted record.
5. The omitted event has no on-chain evidence.

4.4.3 Likelihood

LOW

- The attacker must already have internal backend access.
- The deletion window is short and bounded by the notarizer interval.
- The attack is therefore realistic as an insider threat, but not as an external one.

4.4.4 Impact

HIGH

- Audit completeness would be reduced.
- The system could fail to prove that a security action occurred.
- Insurance or compliance calculations based on the missing event may be incorrect.

4.4.5 Mitigation 1: Sequential Event ID Detection

```
def run_batch(self):
    events = self.queue.drain()
    event_ids = [e.event_id for e in events]

    expected_ids = range(self.min_id, self.max_id + 1)
    if set(event_ids) != set(expected_ids):
        gap_alert = {
            "expected_ids": list(expected_ids),
            "found_ids": event_ids,
            "missing_ids": list(set(expected_ids) - set(event_ids))
        }
        write_to_gap_alerts_json(gap_alert)
```

If an event disappears, the identifier sequence becomes inconsistent and the gap is recorded explicitly.

4.4.6 Mitigation 2: SQLite Full Record

Every AI decision is also persisted in SQLite with instruction, context, reasoning, action, and timestamps. This creates a second evidence source that can be compared against the batch file to detect omissions.

4.4.7 Verification

The audit procedure compares the batch JSON against the SQLite records over the same time window. If the database contains more events than the notarized batch, a tampering investigation is warranted.

4.4.8 Residual Risk

LOW

- The system does not prevent dropping, but it makes the omission observable.
- Gap alerts and database cross-checking create forensic evidence.
- The remaining risk is primarily one of operational response, not technical invisibility.

4.5 T5: Timestamp Manipulation by Miner or Validator

4.5.1 Threat Description

In proof-of-stake or proof-of-work settings, the block producer can influence `block.timestamp` within protocol limits. That flexibility can distort event ordering and weaken time-based policy or evidence checks.

4.5.2 Attack Vector

1. The attacker becomes the block producer for a specific block.
2. The attacker chooses a timestamp that benefits their narrative.
3. A snapshot or report is written with the manipulated time.
4. Evidence appears older or newer than it really is.

4.5.3 Likelihood

LOW

- The attacker must control block production or collude with the producer.
- Timestamp manipulation is bounded by consensus rules.

4.5.4 Impact

MEDIUM

- Evidence chronology becomes less trustworthy.
- Policy deadlines or claim windows may be misrepresented.
- Time-based reasoning can be distorted even if the underlying record remains intact.

4.5.5 Mitigation: Use `block.number` Instead of `block.timestamp`

```
timestamps[claimId] = block.timestamp; // mutable
```

```
blockNumbers[claimId] = block.number; // monotonic
```

Block numbers are not controllable by the block producer in the same way as timestamps and therefore provide a stronger ordering primitive for security-critical evidence.

4.5.6 Current Implementation

The current system still uses `block.timestamp` in some contracts and backend records. This is acceptable for human-readable logs, but not ideal for security-critical sequencing.

4.5.7 Residual Risk

LOW to MINIMAL, once the mitigation is adopted

- Switching to `block.number` is a small code change.
- Functional behavior remains intact.
- The main residual risk is semantic mismatch if humans later interpret block height as wall-clock time.

4.6 T6: Sequence File Corruption During Atomic Write

4.6.1 Threat Description

If the notarizer crashes while updating `sequence.json`, the file could be truncated or partially written. A corrupted sequence file can reset the event counter after restart and create duplicate event IDs.

4.6.2 Attack Vector

1. The notarizer begins saving the latest sequence number.
2. The process crashes mid-write.
3. The sequence file becomes partially written or invalid JSON.
4. On restart, the loader reads the damaged file or falls back to a default value.
5. Duplicate IDs may be generated, breaking the chain of evidence.

4.6.3 Likelihood

LOW

- The failure requires a precise crash or file-system interruption at the write boundary.
- Modern operating systems reduce the probability, but they do not make it impossible.

4.6.4 Impact

HIGH

- Event identifiers may collide.
- Gap detection can become ambiguous.
- The notarization chain may lose its monotonic ordering guarantees.

4.6.5 Mitigation: Atomic Write with `os.replace`

```
def _save_last_id(self, event_id):
    temp_file = "sequence.json.tmp"
    with open(temp_file, "w") as f:
        json.dump({"last_event_id": event_id}, f)
    os.replace(temp_file, "sequence.json")
```

The temporary file is written completely before the rename occurs. `os.replace` makes the final update atomic from the perspective of readers.

4.6.6 Verification

An intentional crash test should show that the system either preserves the old sequence file or promotes the new one in full, never a partial hybrid. After restart, sequence loading should remain valid and event IDs should continue monotonically.

4.6.7 Residual Risk

LOW

- Atomic rename is a standard durability pattern used in production systems.
- The remaining risk is limited to pathological storage failures outside the application model.

4.7 T7: Race Condition in Sequential ID Assignment

4.7.1 Threat Description

If multiple worker processes call the ID allocator concurrently, each may read the same `last_event_id` and emit the same next identifier. This creates duplicate IDs and breaks batch uniqueness.

4.7.2 Attack Vector

1. Process A reads the current sequence value.
2. Process B reads the same sequence value.
3. Both increment it independently.
4. Both write the same next identifier.
5. Two distinct events now share one ID.

4.7.3 Likelihood

MEDIUM

- The problem appears whenever more than one worker process is enabled.
- The threat is accidental as well as adversarial, because a configuration error alone is sufficient.

4.7.4 Impact

MEDIUM

- Duplicate IDs complicate batch processing and proof generation.
- Downstream gap detection may report a false anomaly or miss the true root cause.
- The overall audit trail becomes less reliable.

4.7.5 Mitigation: `asyncio.Lock` and Single-Worker Deployment

```
sequence_lock = asyncio.Lock()
```

```
async def get_next_id(self) -> int:
    async with sequence_lock:
        self.last_event_id += 1
        self._save_last_id(self.last_event_id)
    return self.last_event_id
```

The lock serializes access within one process, while the deployment constraint requires `uvicorn` to run with `workers=1`. Together, these controls preserve sequence uniqueness in the documented deployment model.

4.7.6 Limitation: Single-Worker Constraint

`asyncio.Lock` does not synchronize separate processes. If the server is started with multiple workers, each worker maintains its own lock and memory state, so duplicate IDs can still occur. The README-enforced `workers=1` requirement is therefore a security control, not a mere performance suggestion.

4.7.7 Residual Risk

LOW in the documented deployment model; MEDIUM if the deployment constraint is violated

- The control is effective when the application is run exactly as specified.
- Misconfiguration is the primary residual risk.
- Operational enforcement is required because the code alone cannot prevent multi-process startup.

5 Conclusion

Aegis OS demonstrates a well-structured hybrid trust model: high-value operations are anchored on chain, off-chain reasoning is hashed and preserved, and several important classes of attacks are already mitigated with standard defensive patterns. The strongest controls are the reentrancy guard, domain-separated Merkle hashing, atomic file writes, and the single-worker sequence constraint.

The remaining risks are concentrated in governance hardening, operational discipline, and trust in the backend execution environment. For the current academic deployment, these risks are acceptable because they are visible, bounded, and documented. For any future production deployment, however, the medium-risk items should be treated as mandatory upgrade candidates rather than permanent design choices.